Listing C.1 Maven dependencies

```
<dependencies>
 <dependency>
   <groupId>co.elastic.clients</groupId>
   <artifactId>elasticsearch-java</artifactId>
   <version>8.5.3</version>
 </dependency>
 <dependency>
   <groupId>com.fasterxml.jackson.core</groupId>
   <artifactId>jackson-databind</artifactId>
   <version>2.12.7</version>
 </dependency>
...
</dependencies>
```

I am using the latest and greatest library versions at the time of writing this section: the 8.5.3 version of the elasticsearch-java client artifact and the 2.12.7 version of the Jackson core library. You may need to upgrade these libraries based on your needs.

If you use Gradle, add the following artifacts as dependencies in your Gradle build file.

Listing C.2 Gradle dependencies

```
dependencies {
  implementation 'co.elastic.clients:elasticsearch-java:8.5.3'
  implementation 'com.fasterxml.jackson.core:jackson-databind:2.12.7'
}
```

Once we set up the project, the next step is to initialize the client and get it to work for us.

C.4 Initialization

Let's look at the client initialization and how we can put it to work. The client class is `co.elastic.clients.elasticsearch.ElasticsearchClient` and is initialized by providing the transport object (`co.elastic.clients.transport.ElasticsearchTransport`) to its constructor. This transport object, in turn, needs the `restClient` and JSON mapper objects. Let's look at the steps:

1. Create a `RestClient` object that encapsulates Apache's `HttpHost`, pointing to the URL of the Elasticsearch server.

Listing C.3 Instantiating `RestClient`

```
RestClient restClient = RestClient.builder(
       new HttpHost("localhost", 9200)).build();
```

The `RestClient` builder is created and accepts our Elasticsearch endpoint exposed on localhost at port 9200.